Introduction to Data Processing with Python

This is an introducion to Python for someone with little or no prior experience in programming.

Computers are used for financial processing, scientific calculations, data science and any task that uses *data*, primarily *text* or numbers, and *information*.

You will need to be comfortable with:

- Basic computer skills such as managing files, running programs and using a web browser.
- How to use Jupyter.
- The basic kinds of data, mainly numbers and text.
- Basic math and arithmetic.

Data Processing

Data processing is used in the financial, commercial, healthcare, transportation, and engineering industries. It summarizes, classifies, and interprets raw *data* into organized and analyzed information that can be used for decisions and visualization.

Data processing consists of several steps.

1. Collecting raw data from external sources.
2. Preprocessing or cleaning data to reduce errors, duplications, inconsistencies, and inaccuracies.
3. Storing usable data in files or databases to make it available to analyze.
4. Interpreting data to produce actionable information using methods such as statistics and data science.
5. Providing resulting information for decision making and planning.

Programs

Computers run programs made up of *instructions* that process data, like a recipe.

- Programmers take input data, *algorithms*, and the desired output to write Python programs.
- Python translates Python code into machine instructions that the computer can run.
- Computers *CPUs* run instructions then convert input data to program output.

The advantage of computers is that they can process instructions tirelessly, faster and more accurately than people. Programs are often written to perform manual work that already exists. The effort of writing programs is often repaid by automating tasks that are tedious and require little skill.

Instructions

Instructions read, write, and transform data using logic and calculations for use in later instructions and in creating output from the program. Instructions are written in a readable language that is translated into a form that a computer can run. This will describe the kinds of instructions that a Python program can run, what they do, and how to write them.

Many programs frequency perform the same tasks, so they use *functions* that collect groups of instructions together to save time programming. Functions are much like programs used within programs. Python provides a *library* of common program functions. Much of the effort learning to program is memorizing what these functions do and how to use them in programs.

The following sections describe of basic types of data used in programs and useful functions to process them. The sections use knowledge from previous sections so you are meant to go through them one by one. When you are done, you will be able to write basic Python programs like the examples at the end.

The Internet has a wealth of information to make programming easier and more effective. python.org and w3schools.com are good sources of basic information on using Python. There are many useful books on Python or further study and to use as references. Searching for the type of program you want to write and looking for examples you can use as samples can save you a lot of time programming. Python gives you an error you cannot understand, search for the

error and you may find an explanation or solution on websites like stackexchange.com and stackoverflow.com.

In [8]:
```
print("Hello world")
```

```
Hello world
```

`print` is a function that surrounds some *arguments* with parentheses `(` and `)` and separates them with commas `" , "` and prints them out A function collects the arguments and then runs some instructions that use the argument values. Other functions might give a useful value after running.

There are two ways to print: print text on a complete line like above, or print and stop in the middle of the line, and print the rest after. Run this to print text on separate lines.

In [9]:
```
print("Hello world. ")
print("Have a nice day.")
```

```
Hello world.
Have a nice day.
```

Run this to print text twice on the same line. The `end=""` makes `print` stop without starting a new line.

In [10]:
```
print("Hello world. ", end="")
print("Have a nice day.")
```

```
Hello world. Have a nice day.
```

Strings are text that can be inside either two double quotes `"` or two single quotes `'`. The same quote starts and ends the string.

In [11]:
```
print("this is a double-quoted string")
print('this is a single-quoted string')
```

```
this is a double-quoted string
this is a single-quoted string
```

If you need to put a double quote in a string, you can use a single-quoted string.

In [12]:
```
print("this is a double-quoted string with a single quote ' in it")
```

```
this is a double-quoted string with a single quote ' in it
this is a single-quoted string with a double quote " in it
```

If you need to put a single quote in a string, you can use a double-quoted string.

In [ ]:
```
print('this is a single-quoted string with a double quote " in it')
```

Variables

Programs pass values from instruction to instruction using *variables*. This stores 5 in a variable named `x` and prints it out. `=` assigns 5 to `x`.

In [13]:
```
x = 5
print("x is ", x)
```

```
x is  5
```

The variable can be shown like this, where `x` is a name for a place in which the value 5 is stored.

Another way to print the variable `x` is with a *formatted string*, beginning with `f'` and ending with `'`. This prints out the value of the variable `x` in a formatted string, surrounding the variable `x` with `{` and `}`. Any Python value can be used between `{` and `}`. This can print a variable in a way that is more to your liking.

In [14]:
```
x = 5
print(f'x is {x}.')
```

```
x is 5.
```

A variable can be assigned a value and then be assigned to another value to replace it. This stores 5 in the variable `x`, and prints it out. It then stores 7 in `x` and prints it out.

In [15]:
```
x = 5
print(f'first x is {x}')
x = 7
print(f'then x is {x}')
```

```
first x is 5
then x is 7
```

This can be shown like this, where `x` has the data 5 stored in it that is then replaced with the data 7, where → means "set to new value".

A variable can also be assigned the value of another variable. This stores 5 in the variable `x`, then assigns the variable `y` to the value of `x` and prints it out.

In [16]:
```
x = 5
y = x
print(f'x is {x} and y is {y}')
```

```
x is 5 and y is 5
```

Variables can store text strings as well as numbers. This stores a string in the variable `s` and prints it out.

In [17]:
```
s = "Hello world"
print(f'the text is "{s}"')
```

```
the text is "Hello world"
```

A note on variable names

Variable names start with a *lowercase* letter, `a` to `z`, or an *uppercase* letter, `A` to `Z`, or `_`. They can have a `_`, letter, or number inside but no other punctuation in them. They can not have a number at the beginning.

In [18]:
```
x = 2
x_y = 3
_x = 4
print(f'x is {x}, x_y is {3}, _x is {4}')
```

```
x is 2, x_y is 3, _x is 4
```

Capital and small letters matter, the variable name must always look exactly the same everywhere.

In [19]:
```
X = 5
x = 6
print(f'X is {X}, x is {x}')
```

```
X is 5, x is 6
```

In the code here we will use different variable names for different types of data. They will be these:

- `s`, `t`, `u`, `v`, and so on will have *string* values
- `x`, `y`, `z`, and sometimes `n` will have number values

Data

Numbers and strings are only two types of data in Python. Numbers and strings start as *literal constants* and are the most basic types of data in Python. `"hello"` and `5` are literal constants. Python instructions can combine these values with different *operations* and can store them in variables. Instructions can use literal constants or can use the values stored in variables. Run this to add two values and store them in the variable `y`.

In [20]:
```
x = 5
y = x + 2
print(f'x is {x} and x + 2 is {y}')
```

```
x is 5 and x + 2 is 7
```

The sections below will show examples of operations available for these data types.

Numbers

Numbers in Python are either *integers*, which are signed whole numbers, and *real numbers* that

In [21]:
```
x = 5 + 3.2
print(f'5 + 3.2 is {x}')
```

```
5 + 3.2 is 8.2
```

Number operators

The usual arithmetic operators can be used, like addition `+` and subtraction `-` .

In [22]:
```
x = 5 + 3
print(f'x = 5 + 3 is {x}')
x = 4 - 1
print(f'x = 4 - 1 is {x}')
```

```
x = (4 + 2 - 1) * 3.2 is 16.0
```

You can do multiplication with `*` instead of `x` .

In [ ]:
```
x = 4 * 3.2
print(f'x = 4 * 3.2 is {x}')
```

We do division with `\` instead of $\div$.

In [23]:
```
x = x / 2.0
print(f'x / 2.0 is {x}')
```

```
x / 2.0 is 8.0
```

There are some special operators on numbers. We compute exponentials, say 3^2, like this.

In [24]:
```
x = 3**2
print(f'3**2 is {x}')
```

```
3**2 is 9
```

Another number operator is *modulo*, like "9 modulo 2", which gives the remainder after dividing 9 by 2. You can do modulo operations with `%` .

In [25]:
```
x = 9 % 2
print(f'9 % 2 is {x}')
```

```
9 % 2 is 1
```

Operator shortcuts

Some operators are used to change a variable's value, like `x = x + 1` . There is a shortcut for each of the operators `+` , `-` , `*` , `/` , `**` , and `%` .

For addition, `x = x + 1`, the shortcut is `x += 1`.

In [26]:
```
x = 3
print(f'x starts as {x}')
x += 2
print(f'x += 2 is {x}')
```

```
x starts as 3
"x += 2" is 5
```

For subtraction, `x = x - 1`, the shortcut is `x -= 1`

In [27]:
```
x -= 1
print(f'x -= 1 is {x}')
```

```
"x -= 1" is 4
```

For multiplication, `x = x * 4`, the shortcut is `x *= 4`.

In [28]:
```
x *= 4
print(f'x *= 4 is {x}')
```

```
"x *= 4" is 16
```

For division, `x = x / 1`, the shortcut is `x /= 1`.

In [29]:
```
x /= 4
print(f'x /= 4 is {x}')
```

```
"x /= 4" is 4.0
```

For applying exponents, `x = x ** 2`, the shortcut is `x **= 2`.

In [30]:
```
x **= 2
print(f'x **= 2 is {x}')
```

```
"x **= 2" is 16.0
```

For `x = x + 3`, the shortcut is `x += 3`.

In [31]:
```
x %= 3
print(f'x %= 3 is as {x}')
```

```
"x %= 3" is as 1.0
```

`abs` function

The `abs` function is a function that gives a value when it is done. It takes a number that might be negative and gives the absolute value of it.

In [32]:
```
x = 3
y = -5
z = -4.3
print(f'abs({x}) is {abs(x)}.')
print(f'abs({y}) is {abs(y)}.')
print(f'abs({z}) is {abs(z)}.')
```

```
abs(3) is 3.
abs(-5) is 5.
abs(-4.3) is 4.3.
```

`Max` function

The value of the `max` function is the largest of the numbers give given to it as an argument.

In [33]:
```
L = (1, 12, 9)
print(f'the maximum of {L} is {max(L)}')
```

```
the maximum of (1, 12, 9) is 12
```

`Min` function

The value of the `min` function is the smallest of the numbers give given to it as an argument.

In [34]:
```
L = (1, 12, 9)
print(f'the minimum of {L} is {min(L)}')
```

```
the minimum of (1, 12, 9) is 1
```

`Round` function

The `round` function takes a real number and reduces the size of the fraction. It gives the integer value without the fraction if only the number is given as the argument. If a second argument is a number, that is the size of the fraction to keep.

In [35]:
```
x = 5.34
y = 9.2345
z = -8.431
print(f'round({x}) is {round(x)}.')
print(f'round({y}, 2) is {round(y, 2)}.')
print(f'round({z}) is {round(z)}.')
```

```
round(5.34) is 5.
round(9.2345, 2) is 9.23.
round(-8.431) is -8.
```

Strings

Strings are text that is made up of *characters*, which are letters, numbers, punctuation, and some special characters called *escape characters*. Strings have operations and functions that combine, remove, search, and replace for parts of text. This is the string `"hello"` . Each

character has a *position* in the string, and the positions start counting from `0` rather than `1`.

Escape characters

Escape characters are characters following a *backslash* character `\` in a string. The backslash gives a special meaning to the character that follows it. `\t` means the tab key on the keyboard, and `\n` is the enter key, called a *newline*. Because the backslash `\` changes the meaning of the character that follows it, to really get a backslash you need to use `\\`.

In [36]:

```
print('this is the tab "\t" character')
print('this is the enter "\n" character')
print('this is the backslash "\\" character')
```

```
this is the tab "       " character
this is the enter "
" character
this is the backslash "\" character
```

You can put a double quote in a double-quoted string by using backslash to escape it, an *escaped* double quote, that can go into a double-quoted string but not end the string.

In [37]:

```
print("this is a double-quoted string with a double quote \" in it")
```

```
this is a double-quoted string with a double quote " in it
this is a single-quoted string with a single quote ' in it
```

An escaped single quote can be put in a single-quoted string the same way.

In [ ]:
```
print('this is a single-quoted string with a single quote \' in it')
```

Putting a single quote in the formatted string starting with `f'` and ending with a `'` needs an escaped single quote.

In [38]:
```
print(f'this is a formatted string with a single quote \' in it')
```

```
this is a formatted string with a single quote ' in it
```

Operations on strings

Strings can be combined with the `+` add operator.

In [39]:
```
s = "Again and "
s = s + "again ..."
print(f's is "{s}"')
```

```
s is "Again and again ..."
```

The `+=` shortcut works as well.

In [40]:
```
s = "Again and "
s += "again ..."
print(f's is "{s}"')
```

```
s is "Again and again ..."
```

Strings can be repeated with the `*` multiplication operator.

In [41]:
```
s = "hello " * 3
print(f's is "{s}"')
```

```
s is "hello hello hello "
```

The `*=` shortcut works as well.

In [42]:
```
s = "hello "
s *= 3
print(f's is "{s}"')
```

```
s is "hello hello hello "
```

Characters in strings

Characters in strings can accessed with '[' and ']', where *string* `[` *position* `]` gives the character at *position* in the string. Python operators and functions count character positions from 0 rather than 1. The string can count *position* from the beginning of the string, or if *position* is negative, it can count from the back of the string.

In [43]:

```
s = "A red hat"
print(f's starts as "{s}"')
print("positions:   012345678")
t = s[0]
u = s[3]
v = s[-1]
print(f's[0] is "{t}"')
print(f's[3] is "{u}"')
print(f's[-1] is "{v}"')
```

```
s starts as "A red hat"
positions:   012345678
s[0] is "A"
s[3] is "e"
s[-1] is "t"
```

Slices

Strings that are part of other strings are *slices* or *substrings*. The slice operator looks like *variable* `[` *start* `:` *end* `]` where the slice has all the characters from the character in the *start* position to the character *just before* the *end* position.

A slice that has no *start* starts from the beginning of the string. A slice that has no *end* goes to the end of the string.

In [44]:

```
s = "A red hat"
print(f's starts as "{s}"')
print("positions:   012345678")
t = s[:1]
print(f's[:1] is "{t}"')
t = s[2:5]
print(f's[2:5] is "{t}"')
t = s[6:]
print(f's[6:] is "{t}"')
```

```
s starts as "A red hat"
positions:   012345678
s[:1] is "A"
s[2:5] is "red"
s[6:] is "hat"
```

Len function

The `len` function gives the value of the string length.

In [45]:

```
s = "A red hat"
length = len(s)
print(f'len("{s}") is {length}')
```

```
len("A red hat") is 9
```

Find function

The `find` function gives the first position where you can find a substring in a string.

In [46]:
```
s = "A red hat"
print(f's starts as "{s}"')
print("positions:   012345678")
x = s.find("red")
print(f's.find("red") is {x}')
```

```
s starts as "A red hat"
positions:   012345678
s.find("red") is 2
```

`Count` function

The `count` function will count how many times a substring can be found in a string.

In [47]:
```
s = "the cat in the hat is at the store."
print(f's starts as "{s}"')
x = s.count("the")
print(f's.count("the") is {x}')
x = s.count("at")
print(f's.count("at") {x}')
```

```
s starts as "the cat in the hat is at the store."
s.count("the") is 3
s.count("at") 3
```

`Upper` function

The `upper` function makes all alphabet characters in a string into capitals or *uppercase*.

In [48]:
```
s = "the cat in the hat is at the store"
print(f's starts as "{s}"')
t = s.upper()
print(f's.upper() is "{t}".')
```

```
s starts as "the cat in the hat is at the store"
s.upper() is "THE CAT IN THE HAT IS AT THE STORE".
```

`Lower` function

The `lower` function makes all alphabet characters in a string into small letters or *lowercase*.

In [49]:
```
s = "THE CAT IN THE HAT IS AT THE STORE"
print(f's starts as "{s}"')
t = s.lower()
print(f's.lower() is "{t}"')
```

```
s starts as "THE CAT IN THE HAT IS AT THE STORE"
s.lower() is "the cat in the hat is at the store"
```

`Replace` function

The `replace` function will find the substring of a string that matches a *pattern*, and replace it with another string.

In [50]:
```
s = "the red hat"
print(f's starts as "{s}"')
t = s.replace("red", "blue", 3)
print(f's.replace("red", "blue", 3) is "{t}"')
```

```
s starts as "the red hat"
s.replace("red", "blue", 3) is "the blue hat"
```

`Str` function

If you have a number, the `str` function will let you use it as a string. This is taking str() of an integer.

In [51]:
```
x = 5
s = str(x)
print(f'str({x}) is "{s}"')
```

```
str(5) is "5"
```

This is taking str() of a real number.

In [52]:
```
x = 3.1
s = str(x)
print(f'str({x}) is "{s}"')
```

```
str(3.1) is "3.1"
```

`Int` function

If you have a string but would like to use it as a number, the `int` function will make an integer out of it.

In [53]:
```
s = "5"
x = int(s)
print(f'int("{s}") is {x}')
```

```
int("5") is 5
```

`Float` function

If you have a string but would like to use it as a number, the `float` function will make a real number out of it.

In [54]:
```
s = "3.1"
x = float(s)
print(f'float("{s}") is {x}')
```

```
float("3.1") is 3.1
```

`Input` function

The `input` function can print a string on the console and wait for you to type some text, then give the text you typed as the value.

In [56]:
```
s = input('Enter some text: ')
print(f'The text you typed is "{s}"')
```

```
The text you typed is "hello"
```

`Strip` function

The `strip` function removes a *newline* at the end of a string. Lines read from a text file will end with a newline. Printing a file with a newline will add an extra blank line.

In [55]:
```
s = "line from file\n"
print(s)
print(s.strip())
print("end of printing")
```

```
line from file

line from file
end of printing
```

`Format` function

The `format` function can make output look just the way you want. The format function looks

like this.

`'{: format : format ...}.format( item, item )`

where each *format* is listed below.

This is an example of a `format` output that includes more than one *format*.

```
'{:17s}  ${:7.2f}'.format(name, total)
```

The `:3d` format will make a string out of an integer where the string is at least 3 characters wide, with spaces on the left if the integer is less than 3 characters long.

In [57]:

```
s ='{:3d}'.format(2)
t ='{:3d}'.format(65)
u ='{:3d}'.format(138)
print(f'"{s}" is the number 2 with format ":3d"')
print(f'"{t}" is the number 65 with format ":3d"')
print(f'"{u}" is the number 138 with format ":3d"')
```

```
"  2" is the number 2 with format ":3d"
" 65" is the number 65 with format ":3d"
"138" is the number 138 with format ":3d"
```

The `:<3d` format will make a string out of an integer where the string is at least 3 characters wide, with spaces on the right if the integer is less than 3 characters long.

In [58]:

```
s ='{:<3d}'.format(2)
t ='{:<3d}'.format(65)
u ='{:<3d}'.format(138)
print(f'"{s}" is the number 2 with format "<:3d"')
print(f'"{t}" is the number 65 with format "<:3d"')
print(f'"{u}" is the number 138 with format "<:3d"')
```

```
"2  " is the number 2 with format "<:3d"
"65 " is the number 65 with format "<:3d"
"138" is the number 138 with format "<:3d"
```

The `:^3d` format will make a string out of an integer where the string is at least 3 characters wide but with the number in the middle if the integer is less than 3 characters long.

In [59]:

```
s ='{:^3d}'.format(2)
t ='{:^3d}'.format(65)
u ='{:^3d}'.format(138)
print(f'"{s}" is the number 2 with format "^:3d"')
print(f'"{t}" is the number 65 with format "^:3d"')
print(f'"{u}" is the number 138 with format "^:3d"')
```

```
" 2 " is the number 2 with format "^:3d"
"65 " is the number 65 with format "^:3d"
"138" is the number 138 with format "^:3d"
```

For real numbers, the `:2f` format will make a string out of a real number where the string has

up to 2 numbers in the fraction, with zeros on the right if the fraction of the real number has less than 2 digits

In [60]:

```
s ='{:.2f}'.format(4)
t ='{:.2f}'.format(1.9)
u ='{:.2f}'.format(8.26)
print(f'"{s}" is the 4 with format ":.2f"')
print(f'"{t}" is the 1.9 with format ":.2f"')
print(f'"{u}" is the 8.26 with format ":.2f"')
```

```
"4.00" is the 4 with format ":.2f"
"1.90" is the 1.9 with format ":.2f"
"8.26" is the 8.26 with format ":.2f"
```

For strings, he `:5s` format will make a string at least 5 characters wide but with spaces on the right if the string is less than 3 characters long. The `:>5s` will make the string at least 5 characters long but with spaces on the left, and `^5s` will make the string at least 5 characters long but in the center of the string.

In [61]:

```
s ='{:5s}'.format("one")
t ='{:>5s}'.format("one")
u ='{:^5s}'.format("one")
v ='{:^5s}'.format("eight")
print(f'"{s}" is the "one" with format ":5s"')
print(f'"{t}" is the "one" with format ":>5s"')
print(f'"{u}" is the "one" with format ":^5s"')
print(f'"{v}" is the "eight" with format ":5s"')
```

```
"one  " is the "one" with format ":5s"
"  one" is the "one" with format ":>5s"
" one " is the "one" with format ":^5s"
"eight" is the "eight" with format ":5s"
```

Compound Data Types

There are *compound data types* that are collections of multiple data values that can be assigned to variables. Two of these are *lists* and *dictionaries*, and are used to store things like grocery lists and telephone books that collect items of basic data types together.

Variable names for compound data types will be:

- `L`, `M`, `N`, and so on will be lists
- `D`, `E`, `F`, and so on will be dictionaries

Lists

The simplest compound data type are *lists*. Lists collect a series of other data type values in order between square brackets `[` and `]`.

In [62]:
```
L = [5, 9, 1]
print(f'L is {L}')
```

```
L is [5, 9, 1]
```

The list can be shown like this.

Operations on lists

Lists can be combined with the `+` add operator.

In [63]:
```
L = [5, 9, 10]
L = L + [4, 7]
print(f'[5, 9, 10] + [4, 7] is {L}')
```

```
[5, 9, 10] + [4, 7] is [5, 9, 10, 4, 7]
```

The `+=` shortcut works as well.

In [64]:
```
L = [5, 9, 10]
L += [4, 7]
print(f'[5, 9, 10] + [4, 7] is {L}')
```

```
[5, 9, 10] + [4, 7] is [5, 9, 10, 4, 7]
```

Lists can be repeated with the `*` multiplication operator.

In [65]:
```
L = [1, 2] * 3
print(f'[1, 2] * 3 is {L}')
```

```
[1, 2] * 3 is [1, 2, 1, 2, 1, 2]
```

The `*=` shortcut works as well.

In [66]:
```
L = [4, 5]
L *= 2
print(f'[4, 5] * 2 is {L}')
```

```
[4, 5] * 2 is [4, 5, 4, 5]
```

Items in lists

Each list item value can be accessed with `[` and `]`, where *list* `[` *position* `]` gives the item at *position* in the list. List operators and functions count item positions from 0 rather than 1. The list can count *position* from the beginning of the list or if *position* is negative, it can count from the back of the list.

In [67]:

```
L = [5, 9, 1]
x = L[0]
y = L[-2]
z = L[2]
print(f'L is {L}')
print(f'L[0] is {x}')
print(f'L[-2] is {y}')
print(f'L[2] is {z}')
```

```
L is [5, 9, 1]
L[0] is 5
L[-2] is 9
L[2] is 1
```

The value of an list item can be changed in the list by assigning an item with its position in square brackets `[` and `]`. Like before, positions start from 0.

In [68]:

```
L = [5, 9, 1]
print(f'L starts as {L}')
L[0] = 4
L[-2] = 6
L[2] = 2
print(f'L after changing is {L}')
```

```
L starts as [5, 9, 1]
L after changing is [4, 6, 2]
```

Lists and variables

There is not room to store the whole list in a variable, so the variable stores a *pointer* to the list. That can be shown like this.

When a list is assigned to a variable and that variable is assigned to another variable, the entire list isn't copied into the second variable. Only the *pointer* is copied.

We can show that when both variables are pointing to the same list, when we change the list through the first variable we will see the change when looking through the second variable.

In [69]:

```
L = [5, 9, 1]
print(f'L starts as {L}.')
M = L
print(f'after M = L, M is {M}.')
L[1] = 2
print(f'after L[1] = 2, L is {L} and M is {M}.')
```

```
L starts as [5, 9, 1].
after M = L, M is [5, 9, 1].
after L[1] = 2, L is [5, 2, 1] and M is [5, 2, 1].
```

`Copy` function

To assign a list to another variable and be able to change the list through the second variable without affecting the first list, the `copy` function makes a copy of the list before assigning it to the second variable. Changing the second list then doesn't change the first list.

In [70]:

```
L = [5, 9, 1]
print(f'L starts as {L}.')
M = L.copy()
print(f'after M = L, M is {M}.')
L[1] = 2
print(f'L after L[1] = 2, L is {L} and M is {M}.')
```

```
L starts as [5, 9, 1].
after M = L, M is [5, 9, 1].
L after L[1] = 2, L is [5, 2, 1] and M is [5, 9, 1].
```

Slices

Lists that are part of other lists are *slices* or *sublists*. The slice operator looks like *variable* `[` *start* `:` *end* `]` where the slice are all the items from the item in the *start* position to the item *just before* the *end* position.

A slice that has no *start* starts from the beginning of the list. A slice that has no *end* goes to the end of the list.

In [71]:

```
L = [5, 19, 1, 10, 8, 21]
print(f'L is {L}')
x = L[:1]
print(f'L[0:1] is {x}')
x = L[:3]
print(f'L[:3] is {x}')
x = L[4:]
print(f'L[4:] is {x}')
```

```
L is [5, 19, 1, 10, 8, 21]
L[0:1] is [5]
L[:3] is [5, 19, 1]
L[4:] is [8, 21]
```

Len function

The `len` function gives the value of the list length.

In [72]:

```
L = [5, 9, 1]
x = len(L)
print(f'len({L}) is {x}')
```

```
len([5, 9, 1]) is 3
```

Append function

New items can be added at the end of the list with the `append` function.

In [73]:

```
L = [5, 9, 1]
print(f'L starts as {L}')
L.append(15)
print(f'after L.append(15), L is {L}')
```

```
L starts as [5, 9, 1]
after L.append(15), L is [5, 9, 1, 15]
```

When the `append` function is called for a list, the way it is called is *list*. `append(` *item* `)`. The list is still given to `append` as an argument even though it is not passed between `(` and `)`. When a list is given as an argument to a function, the pointer is given to the function. Those functions then can use the pointer to change the list, then the list will still be changed when the function is done.

`Insert` function

Items can be added in the beginning or the middle of the list with the `insert` function. The position where the item is added is the first argument to `insert`, starting from 0, and the item to add is the second argument. All the items after the one added are moved down the list.

In [74]:

```
L = [5, 9, 1]
print(f'L starts as {L} and the length is {len(L)}. L[2] is {L[2]}.')
L.insert(2, 4)
print(f'L.insert(2, 4) is {L} and the length is {len(L)}. L[2] is {L[2]}.')
```

```
L starts as [5, 9, 1] and the length is 3. L[2] is 1.
L.insert(2, 4) is [5, 9, 4, 1] and the length is 4. L[2] is 4.
```

This shows `L.insert(2, 4)`.

In [75]:

```
L = [5, 9, 1]
print(f'L starts as {L} and the length is {len(L)}. L[2] is {L[2]}.')
L.insert(0, 7)
print(f'L.insert(0, 7) is {L} and the length is {len(L)}. L[2] is {L[2]}.')
```

```
L starts as [5, 9, 1] and the length is 3. L[2] is 1.
L.insert(0, 7) is [7, 5, 9, 1] and the length is 4. L[2] is 9.
```

This shows `L.insert(0, 7)` .

Pop function

An item at some position can be removed with the `pop` function. The position of the item to remove is the first argument to `pop` , starting from 0. All items after the one removed are moved up the list. The item removed is the value of `pop` .

In [76]:

```
L = [5, 9, 1]
print(f'L starts as {L} and the length is {len(L)}.')
x = L.pop(1)
print(f'L after L.pop(1) is {L} and the length is {len(L)}. The item removed i
```

```
L starts as [5, 9, 1] and the length is 3.
L after L.pop(1) is [5, 1] and the length is 2. The item removed is 9.
```

This shows `L.pop(1)` , which gives the 9 removed as the value.

`Remove` function

The first item having some value can be removed from the with the `remove` function. The value of the item to remove is the first argument to `remove`. Only the first item having the value will be removed. All items after the one removed are moved up the list.

In [145…

```
L = [5, 9, 1, 9]
print(f'L starts as {L} and the length is {len(L)}')
L.remove(9)
print(f'L after L.remove(9) is {L} and the length is {len(L)}.')
```

```
L starts as [5, 9, 1, 9] and the length is 4
L after L.remove(9) is [5, 1, 9] and the length is 3.
```

This is removing `9` a second time.

In [146…

```
L.remove(9)
print(f'L after L.remove(9) is {L} and the length is {len(L)}.')
```

```
L after L.remove(9) is [5, 1] and the length is 2.
```

This shows calling `L.remove(9)` twice.

`Extend` function

A list can be added to the end of a list with the `extend` function, like the `+` operator. The list to add at the end of the first list is the argument to `extend`.

In [78]:

```
L = [5, 9, 1]
print(f'L starts as {L} and the length is {len(L)}')
M = [2, 4]
L.extend(M)
print(f'after L.extend(M), M is {M}, L is {L} and len(L) is {len(L)}.')
```

```
L starts as [5, 9, 1] and the length is 3
after L.extend(M), M is [2, 4], L is [5, 9, 1, 2, 4] and len(L) is 5.
```

This shows `L.extend(M)`.

`Sum` , `Min` , `Max` functions

Lists that are just numbers have functions to give the sum, minimum, and maximum.

In [79]:

```
L = [4, 1, 8, 2]
print(f'sum({L}) = {sum(L)}')
print(f'min({L}) = {min(L)}')
print(f'max({L}) = {max(L)}')
```

```
sum([4, 1, 8, 2]) = 15
min([4, 1, 8, 2]) = 1
max([4, 1, 8, 2]) = 8
```

`Sort` function

Lists can be sorted with the `sort` function into increasing number or alphabetic order. This shows sorting a list of numbers.

In [80]:

```
L = [4, 1, 8, 2]
print(f'L starts as = {L}')
L.sort()
print(f'after L.sort() L = {L}')
```

```
L starts as = [4, 1, 8, 2]
after L.sort() L = [1, 2, 4, 8]
```

This shows `L` after `L.sort()` .

This shows sorting a list of strings.

In [81]:

```
L = ["horse", "cat", "ox", "dog"]
print(f'L starts as = {L}')
L.sort()
print(f'after L.sort() L = {L}')
```

```
L starts as = ['horse', 'cat', 'ox', 'dog']
after L.sort() L = ['cat', 'dog', 'horse', 'ox']
```

This shows `L` after `L.sort()` .

`Reverse` function

Lists items can be reversed with the `reverse` function.

In [82]:

```
L = [4, 1, 8, 2]
print(f'L starts as = {L}')
L.reverse()
print(f'after L.reverse() L = {L}')
```

```
L starts as = [4, 1, 8, 2]
after L.reverse() L = [2, 8, 1, 4]
```

This shows `L` after `L.reverse()` .

Lists from strings: `Split` function

Lists can be created from strings with the `split` function. Using `split` with no arguments splits a string at spaces.

In [83]:

```
s = "The cat in the hat"
print(f's starts as = "{s}"')
L = s.split()
print(f'after s.split() L is {L}')
```

```
s starts as = The cat in the hat
after s.split() L is ['The', 'cat', 'in', 'the', 'hat']
```

`split` can split strings separated by a particular string given as a second argument.

In [84]:

```
s = "first,second,third"
print(f's starts as = {s}')
L = s.split(",")
print(f'after s.split(",") L is {L}')
```

```
s starts as = first,second,third
after s.split(",") L is ['first', 'second', 'third']
```

Strings from lists: `join` function

Strings can be created from list items by joining the list items separated by a particular string.

In [85]:

```
L = ["first", "second", "third"]
print(f'L starts as {L}')
s = " ".join(L)
print(f'" ".join(L) is "{s}"')
L = ["O", "N", "E"]
print(f'L starts as {L}')
s = "".join(L)
print(f'"".join(L) is "{s}"')
```

```
L starts as ['first', 'second', 'third']
" ".join(L) is "first second third"
L starts as ['O', 'N', 'E']
"".join(L) is "ONE"
```

This shows `" ".join(L)` .

Dictionaries

Dictionaries are a compound data type similar to *lists*. In a dictionary you look up a value using another value, a *key*, rather than looking up a value by position as in a list. This is like a real dictionary, you look up a definition of a word using the word.

In [86]:

```
D = {"leaf": "green", "sky": "blue", "apple": "red"}
print(f'D is {D}')
```

```
D is {'leaf': 'green', 'sky': 'blue', 'apple': 'red'}
```

The dictionary can be shown like this.

Items in dictionaries

Each dictionary item can be accessed with '[' and ']', where *dictionary* `[` *key* `]` gives the value that is assigned for that key in the dictionary.

In [87]:

```
D = {"leaf": "green", "sky": "blue", "apple": "red"}
leaf = D["leaf"]
apple = D["apple"]
print(f'D["leaf"] is {leaf}')
print(f'D["apple"] is {apple}')
```

```
D["leaf"] is green
D["apple"] is red
```

The value of a dictionary item for a key can be changed by assigning that item with the key in square brackets `[` and `]`.

In [88]:
```
D["apple"] = "honeycrisp"
print(f'after D["apple"] = "honeycrisp", D["apple"]  is "{D["apple"]}"')
```

```
after D["apple"] = "honeycrisp", D["apple"]  is "honeycrisp"
```

New dictionary items can be created by assigning a value with a new key in square brackets `[` and `]`.

In [89]:
```
D = { "leaf": "green", "sky": "blue", "apple": "red"}
print(f'D starts as {D} length {len(D)}')
D["earth"] = "brown"
print(f'after D["earth"] = "brown", D is {D} length {len(D)}')
```

```
D starts as {'leaf': 'green', 'sky': 'blue', 'apple': 'red'} length 3
after D["earth"] = "brown", D is {'leaf': 'green', 'sky': 'blue', 'apple': 're
d', 'earth': 'brown'} length 4
```

Dictionaries and variables

There is not room to store whole dictionary in a variable, so the variable stores a *pointer* to the dictionary. That can be shown like this.

When a dictionary is assigned to a variable and that variable is assigned to another variable, the entire dictionary isn't copied into the second variable. Only the pointer is copied.

We can show that when both variables are pointing to the same dictionary, when we change the dictionary through the first variable, and we will se the change looking through the second variable.

In [90]:
```
D = { "leaf": "green", "sky": "blue", "apple": "red"}
print(f'D starts as {D}.')
E = D
print(f'after E = D, E is {E}.')
D["apple"] = "yellow"
print(f'after D["apple"] = "yellow", D is {D} and E is {E}.')
```

```
D starts as {'leaf': 'green', 'sky': 'blue', 'apple': 'red'}.
after E = D, E is {'leaf': 'green', 'sky': 'blue', 'apple': 'red'}.
after D["apple"] = "yellow", D is {'leaf': 'green', 'sky': 'blue', 'apple': 'y
ellow'} and E is {'leaf': 'green', 'sky': 'blue', 'apple': 'yellow'}.
```

`Copy` function

To assign a dictionary to another variable and be able to change the dictionary through the second variable without affecting the first dictionary , the `copy` function makes a copy of the dictionary before assigning it to the second variable. Changing the second dictionary then doesn't change the first dictionary.

In [91]:

```
D = { "leaf": "green", "sky": "blue", "apple": "red"}
print(f'D starts as {D}')
E = D.copy()
print(f'after E = D.copy(), E is {E}.')
D["apple"] = "yellow"
print(f'after D["apple"] = "yellow", D is {D} and E is {E}')
```

```
D starts as {'leaf': 'green', 'sky': 'blue', 'apple': 'red'}
after E = D.copy(), E is {'leaf': 'green', 'sky': 'blue', 'apple': 'red'}.
after D["apple"] = "yellow", D is {'leaf': 'green', 'sky': 'blue', 'apple': 'y
ellow'} and E is {'leaf': 'green', 'sky': 'blue', 'apple': 'red'}
```

`Len` function

The `len` function has the value of the dictionary length.

In [92]:
```
D = {"leaf": "green", "sky": "blue", "apple": "red"}
print(f'D starts as {D}')
x = len(D)
print(f'{len(D)} is {x}')
```

```
D starts as {'leaf': 'green', 'sky': 'blue', 'apple': 'red'}
3 is 3
```

Keys function

The `keys` function gives a list of the dictionary keys. The keys are not actually in `list` form, you need the `list` function to convert them to a list.

In [93]:
```
D = {"leaf": "green", "sky": "blue", "apple": "red"}
print(f'D starts as {D}')
L = list(D.keys())
print(f'D.keys() is {L}')
```

```
D starts as {'leaf': 'green', 'sky': 'blue', 'apple': 'red'}
D.keys() is ['leaf', 'sky', 'apple']
```

Pop function and `del` operator

The `pop` function can remove an item from a dictionary. The key of the value to remove is the first argument to `pop`. The dictionary is one value smaller after `pop`. The `del` operator can remove an item too.

In [94]:
```
D = {"leaf": "green", "sky": "blue", "apple": "red"}
print(f'D starts as {D}')
D.pop("leaf")
print(f'after D.pop("leaf"), D is {D}')
del D["sky"]
print(f'after del D["sky"], D is {D}')
```

```
D starts as {'leaf': 'green', 'sky': 'blue', 'apple': 'red'}
after D.pop("leaf"), D is {'sky': 'blue', 'apple': 'red'}
after del D["sky"], D is {'apple': 'red'}
```

Instruction control

Programs may not want to run instructions just in the order they are given. You can run instructions only if something is true, repeat instructions while something is true, or for each item in a group. Examples are `if`, `while`, and `for` statements.

`If` statements

An `if` statement can run instructions if something is true. An `if` statement looks like:

`if` *condition* `:` \ *some instruction* \ *another instruction* \ *and more...* \ *instruction run*

after previous instructions

An `if` statment tests a *condition*, like `x < 5` , and if true runs the indented instructions after it. The next un-indented instruction is run whether or not the `if` *condition* is true. The indents have to be one or more spaces, and must be the same for each instruction. Here the first test is true and a line is printed. The second test is not true so no line is printed for that. The last line is always printed.

In [95]:

```
x = 5
if x > 3:
    print("x is more than 3")
if x < 2:
    print("x is less than 2")
print("after x tests")
```

```
x is more than 3
after x tests
```

Conditions

Tests for `if` statements may use different operators or functions. An operator for a condition might be `>` in `x > 3` .

In [147…

```
x = 5
if x > 3:
    print(f'x is > 3')
```

```
x is > 3
```

In other cases a function might be used in the condition.

In [148…

```
s = "hello"
if len(s) > 3:
    print(f'len(s) is > 3')
```

```
len(s) is > 3
```

Combining conditions

Sometimes a condition might want to test more than one thing. An example is wanting a number to be between a lower and higher value. The `and` operator can do more than one test at a time.

In [97]:

```
x = 4
print(f'x starts as {x}')
if x > 2 and x < 6:
    print(f'{x} is between 2 and 6')
if x > 5 and x < 9:
    print(f'{x} is between 5 and 9')
```

```
x starts as 4
4 is between 2 and 6
```

The `or` operator can test if either one test or another is true. Here the second test is true because when x is 4, it is less than 9 even if it is no more than 5.

In [98]:

```
x = 4
print(f'x starts as {x}')
if x > 2 or x < 6:
    print(f'{x} is more than 2 or less than 6')
if x > 5 or x < 9:
    print(f'{x} is more than 5 or less than 9')
```

```
x starts as 4
4 is more than 2 or less than 6
4 is more than 5 or less than 9
```

The `not` operator can test whether a condition is false.

In [99]:

```
x = 3
print(f'x starts as {x}')
if x > 2:
    print('x {x} is greater than 2')
if not x < 1:
    print('x {x} is not less than 1')
```

```
x starts as 3
x {x} is greater than 2
x {x} is not less than 1
```

Tests with numbers

There are lots of condition operators for numbers. `==` tests if numbers are the same.

In [100…

```
s = input("type a number to test:")
x = int(s)
if x == 3:
    print("{x} is 3")
```

```
{x} is 3
{x} is greater than or equal to 3
{x} is less than or equal to 3
```

> tests if a number is more than another.

In [ ]:

```
s = input("type a number to test:")
x = int(s)
if x > 3:
    print("{x} is greater than 3")
```

< tests if a number is less than another.

In [ ]:

```
s = input("type a number to test:")
x = int(s)
if x >= 3:
    print("{x} is greater than or equal to 3")
```

>= tests if a number is the same or more than another.

In [ ]:

```
s = input("type a number to test:")
x = int(s)
if x < 3:
    print("{x} is less than 3")
```

<= tests if a number is the same or less than another.

In [ ]:

```
s = input("type a number to test:")
x = int(s)
if x <= 3:
    print("{x} is less than or equal to 3")
```

!= tests if numbers are not the same.

In [ ]:

```
s = input("type a number to test:")
x = int(s)
if x != 3:
    print("{x} is not 3")
```

Tests with strings

The == operator can test whether strings are the same.

In [150…

```
s = input(f'type a string to test against "test":')
t = "test"
if s == t:
    print(f'"{s}" is "{t}"')
```

```
"test" is "test"
```

The `!=` operator can test whether strings are different.

In [ ]:

```
s = input(f'type a string to test against "test":')
t = "test"
if s != t:
    print(f'"{s}" is not "{t}"')
```

The `in` operator can test whether a string is part of another.

In [ ]:

```
s = input(f'type a string to test against "test":')
t = "test"
if t in s:
    print(f'"{s}" is part of "{t}"')
```

There a number of tests for the types of characters in a string using functions.

`isalpha(` *string* `)` tests whether all characters in a string are letters between "a" and "z" or between "A" and "Z".

In [102…

```
s = input(f'type a string to test:')
if s.isalpha():
    print(f'"{s}" has all letters.')
```

```
"abc123" has all numbers or letters.
```

`isdigit(` *string* `)` tests whether all characters in a string are numbers.

In [ ]:

```
s = input(f'type a string to test:')
if s.isdigit():
    print(f'"{s}" has all digits.')
```

`isalnum(` *string* `)` tests whether all characters in a string are numbers or letters.

In [ ]:

```
s = input(f'type a string to test:')
if s.isalnum():
    print(f'"{s}" has all numbers or letters.')
```

`isspace(` *string* `)` tests whether all characters in a string are spaces .

In [ ]:

```
s = input(f'type a string to test:')
if s.isspace():
    print(f'"{s}" has all spaces.')
```

Tests with lists

The `in` operator tests if an item is in a list.

In [103…

```
L = [1, 3, 9]
print(f'L starts as {L}')
if 3 in L:
    print(f'3 is in {L}')
```

```
L starts as [1, 3, 9]
3 is in [1, 3, 9]
```

Tests with dictionary

The `in` operator tests if a key is in a dictionary.

In [104…

```
D = {"sky": "blue", "leaf": "green"}
print(f'D starts as {D}')
if "sky" in D:
    print(f'"sky" is in {D}')
```

```
D starts as {'sky': 'blue', 'leaf': 'green'}
"sky" is in {'sky': 'blue', 'leaf': 'green'}
```

`Else` statements

You can test both whether a condition is true or false in one `if` statement with `else`. The instructions after the `if` are run if the condition is true and the instructions after `else` are run if the condition is not true.

`if` *condition* `:` \ instruction\ instruction\ *`else:`* \ instruction\ instruction*

In [105…

```
s = input(f'type a string to test:')
if s.isalpha():
    print(f'isalpha("{s}") are all letters.')
else:
    print(f'isalpha("{s}") is not all letters.')
```

```
isalpha("abcdef") are all letters.
```

`Elif` statements

You can test a number of conditions in one `if` statement with `elif`. `elif` lets you test one condition after another. The instructions after the `if` are run if the condition is true and runs the instructions after the first `elif` when that condition is true. There can be an `else` at the end that runs if none of the conditions are true.

`if` *condition* `:` \ *instruction*\ *instruction*\ `elif` *condition* `:` \ *instruction*\ *instruction*\ `elif` *condition* `:` \ *instruction*\ *instruction*\ `else:` \ *instruction*\ *instruction*

In [106…

```
s = input("type a number to test:")
x = int(s)
if x > 9:
    print("{x} is greater than 9")
elif x >= 3:
    print("{x} is between 3 and 9")
else:
    print("{x} is less than 3")
```

```
{x} is between 3 and 9
```

While loop

`if` statements let you run some instructions one time if some condition is true. `while` statements let you run some instructions multiple times until a condition is false. A while loop looks like:

`while` *condition* `:` \ *instruction*\ *instruction*

There is one catch: the condition has to use some variables, and the instructions have to change the value of the variables so that at some time the condition becomes false. Otherwise the program will run those instructions forever. In this loop, `i` starts below 6, but goes up 1 each time so that it eventually becomes 6 and the loop stops.

In [107…

```
x = 2
while x < 6:
    print(f'x is {x}')
    x += 1
```

```
x is 2
x is 3
x is 4
x is 5
```

This `while` loop can convert pounds to kilograms until you enter 0.

In [108…

```
s = ""
while s != "0":
    s = input("give the number of pounds to convert to kilograms, type 0 to s
    x = int(s)
    print(f'{x} pounds is {x * 2.2} kilograms')
```

```
2 pounds is 4.4 kilograms
```

```
0 pounds is 0.0 kilograms
```

`For` loop

The `for` loop is like a `while` loop except that it runs instructions once for each item in a group. A `for` loop looks like this.

`for` *item* `in` *group* `:` \ *instruction*\ *instruction*

In [2]:

```
print("a for loop for a string")
s = "abcd"
for c in s:
    print(f'the next character of "{s}" is "{c}"')
```

```
a for loop for a string
the next character of "abcd" is "a"
the next character of "abcd" is "b"
the next character of "abcd" is "c"
the next character of "abcd" is "d"
```

This `for` loop runs for each item in a list.

In [1]:

```
print("a for loop for a list")
L = ["first", "second", "third"]
for i in L:
    print(f'the next item in "{L}" is "{i}"')
```

```
a for loop for a list
the next item in "['first', 'second', 'third']" is "first"
the next item in "['first', 'second', 'third']" is "second"
the next item in "['first', 'second', 'third']" is "third"
```

Range function

loops often run for each number in a group of numbers. The `range` function creates a list of numbers for the `for` loop to run on. A `for` loop using `range(n)` with one argument loops through the numbers `0` to `n-1`.

In [110…

```
for i in range(4):
    print(f'the next number in range(4) is {i}')
```

```
the next number in range(4) is 0
the next number in range(4) is 1
the next number in range(4) is 2
the next number in range(4) is 3
```

Using `range(m, n)` with two arguments will loop through the numbers `m` to `n-1`.

In [111…

```
for i in range(2, 5):
    print(f'the next number in range(2, 5) is {i}')
```

```
the next number in range(2, 5) is 2
the next number in range(2, 5) is 3
the next number in range(2, 5) is 4
```

Using `range(m, n, step)` can add `step` each time until the number is more than `n-1`.

In [112…

```
for i in range(3, 9, 2):
    print(f'the next number in range(3, 9, 2) is {i}')
```

```
the next number in range(3, 9, 2) is 3
the next number in range(3, 9, 2) is 5
the next number in range(3, 9, 2) is 7
```

Using `range(m, n, -1)` the loop can count down from `m` to `n+1`.

In [113…

```
for i in range(5, 2, -1):
    print(f'the next number in range(5, 2) is {i}')
```

```
the next number in range(5, 2) is 5
the next number in range(5, 2) is 4
the next number in range(5, 2) is 3
```

List comprehensions

Lists can be made using `for` loops. A *list comprehension* has a `for` loop inside `[` and `]` to create the items in a list. List comprehensions look like:

`[` *formula* `for` *variable* `in` *group* `]`

where the *formula* gives a value using the *variable* and the *variable* has the value of each item in the group.

In [114…

```
L = [i for i in range(5)]
print(f'[i for i in range(5)] produces {L}')
L = [2*i for i in range(5)]
print(f'[2*i for i in range(5)] produces {L}')
L = [c for c in "abc"]
print(f'[c for c in "abc"] produces {L}')
```

```
[i for i in range(5)] produces [0, 1, 2, 3, 4]
[2*i for i in range(5)] produces [0, 2, 4, 6, 8]
[c for c in "abc"] produces ['a', 'b', 'c']
```

If the formula does not use the variable, the list will just repeat the value of the formula.

In [115…

```
L = [1 for i in range(5)]
print(f'[1 for i in range(5)] produces {L}')
```

```
[1 for i in range(5)] produces [1, 1, 1, 1, 1]
```

List comprehensions can make creating many different kinds of lists easier because any formula or `for` loop can be used.

Dictionary comprehensions

Dictionaries can be made using `for` loops. A *dictionary comprehension* has a `for` loop inside `{` and `}` to create the items in a dictionary. Dictionary comprehensions look like:

`[` *key*: *value* `for` *variable* `in` *group* `]`

where the *key* gives a value for the key and the *value* gives the key value, either or both using the *variable*.

In [116…

```
keys = ["sky", "leaf"]
print(f'keys starts as {keys}')
values = ["blue", "green"]
print(f'keys starts as {keys}')
colors = {keys[i]: values[i] for i in range(len(keys))}
print('{keys[i]: values[i] for i in range(len(keys))} is ', end="")
print(f'{colors}')
```

```
keys starts as ['sky', 'leaf']
keys starts as ['sky', 'leaf']
{keys[i]: values[i] for i in range(len(keys))} is {'sky': 'blue', 'leaf': 'gre
en'}
```

Dictionary comprehensions can make creating many different kinds of dictionaries easier because any formula or `for` loop can be used.

Functions

When a group of instructions need to be used many times in programs, *functions* save time in programming. Functions can take values as *arguments*, and return values if needed. Python offers a library of useful functions. Functions are defined with `def`. This is an example of a function with no arguments.

In [117…

```
def hello():
    print("hello")
print(f'hello() does this:')
hello()
```

```
hello() does this:
hello
```

This is an example of a function with arguments.

In [118…

```
def print_this(s):
    print(s)
print(f'print_this("hello") does this:')
print_this("hello")
```

```
print_this("hello") does this:
hello
```

This is an example of a function that returns a value.

In [119…

```
def increase(x):
    return x+1
print(f'increase(5) gives this: {increase(5)}')
```

```
increase(5) gives this: 6
```

Variables inside a function only matter inside the function. Variables with the same name inside a function and in the instructions that call them are different.

In [120…

```
def variable_test():
    x = 3
    print(f'in variable_test, after x = 3, x is {x}')
x = 5
print(f'x starts as {x}')
variable_test()
print(f'in the main program after variable_test(), x is {x}')
```

```
x starts as 5
in variable_test, after x = 3, x is 3
in the main program after variable_test(), x is 5
```

Regular Expressions

Regular expressions are *patterns* that match parts of strings. A regular expression can tell if a string matches a pattern.

The variable name `p` will be used to store patterns.

In [121…

```
import re
s = "this needs to be matched"
print(f's starts as "{s}"')
p = "match"
print(f'p starts as "{p}"')
if re.search(p, s):
    print(f're.search(p, s) is True')
```

```
s starts as "this needs to be matched"
p starts as "match"
re.search(p, s) is True
```

A pattern is made of normal string keyboard characters and escape characters, but some

characters and escape characters have special meaning in patterns.

- period `.` matches any character
- the escape character `\.` is needed to match a period, because a period has a special meaning
- the escape character `\d` matches any number character, or *digit*, 0 to 9
- the escape character `\D` is the opposite of `\d` and matches anything that is not a digit
- the escape character `\w` matches any digit or alphabet character a to z, or A to Z
- the escape character `\W` is the opposite of `\w` and matches anything that is not a digit or alphabet character
- the escape character `\s` matches any *space character* which are spaces `" "`, tabs `\t`, or newlines `\n`
- the escape character `\S` is the opposite of `\s` and matches anything that is not a space character

In [122…

```
import re
s = "there are 12 months in the year"
print(f's starts as "{s}"')
p = "\d\d"
print(f'p starts as "{p}"')
if re.search(p, s):
    print(f're.search(p, s) is True')
```

```
s starts as "there are 12 months in the year"
p starts as "\d\d"
re.search(p, s) is True
```

There are special characters that control how characters match in a pattern.

- plus `+` means the normal character or escape character just before it will match one or more of that character in the string
- star `*` means the normal character or escape character just before it will match zero or

more of that character in the string

- question mark `?` means the normal character or escape character just before it will match if that character is in the string but will still match if it is missing
- caret `^` at the beginning of the pattern means the pattern must match starting at the beginning of the string
- dollar sign `$` at the end of the pattern means the pattern must matsh ending at the end of the string
- to match a plus, star, question mark, caret, or dollar sign, the escape character version must be used, `\+` , `\*` . `\?` , `\^` , or `\$`

In [123…

```
import re
s = "there are 12 months in the year"
print(f's starts as "{s}"')
p = "\d+"
print(f'p starts as "{p}"')
if re.search(p, s):
   print(f're.search(p, s) is True')
```

```
s starts as "there are 12 months in the year"
p starts as "\d+"
re.search(p, s) is True
```

The pattern to match a date like '*month*/*day*/*year*, where month and day are two digits like "03/05/2011", is `\d\d/\d\d/\d\d\d\d` , where the `\d` are escape characters and slash `/` matches a normal slash.

In [124…

```
import re
s = "03/05/2011"
print(f's starts as "{s}"')
p = "\d\d/\d\d/\d\d\d\d"
print(f'p starts as "{p}"')
if re.search(p, s):
   print(f're.search(p, s) is True')
```

```
s starts as "03/05/2011"
p starts as "\d\d/\d\d/\d\d\d\d"
re.search(p, s) is True
```

There are special characters to control how many times characters will match in a pattern.

- `{` *count* `}` will match the character just before it *count* times
- `{` *min-count* `,` *max-count* `}` will match the character just before it at least *min-count* times but at most *max-count* times The pattern to match a date can be `\d{2}/\d{2}/\d{4}` this way.

In [125…

```
import re
s = "03/05/2011"
print(f's starts as "{s}"')
p = "\d{2}/\d{2}/\d{4}"
print(f'p starts as "{p}"')
if re.search(p, s):
   print(f're.search(p, s) is True')
```

```
s starts as "03/05/2011"
p starts as "\d{2}/\d{2}/\d{4}"
re.search(p, s) is True
```

Groups of characters and escape characters can match together using `(` and `)`.

In [126…

```
import re
s = "banana"
print(f's starts as "{s}"')
p = "(an){2}"
print(f'p starts as "{p}"')
if re.search(p, s):
   print(f're.search(p, s) is True')
```

```
s starts as "banana"
p starts as "(an){2}"
re.search(p, s) is True
```

The `|` can be used in a group inside `(` and `)` to let either of two patterns to match.

In [127…

```
import re
s = "this has been a long day"
print(f's starts as "{s}"')
p = "a (long|short) day"
print(f'p starts as "{p}"')
if re.search(p, s):
  print(f're.search(p, s) is True')
```

```
s starts as "this has been a long day"
p starts as "a (long|short) day"
re.search(p, s) is True
```

When you match a pattern with escape characters, you can get the exact part of the string matched with the `group` function.

In [128…

```
import re
s = "today 03/05/2011 is March 5th, 2011"
print(f's starts as "{s}"')
p = "\d{2}/\d{2}/\d{4}"
print(f'p starts as "{p}"')
m = re.search(p, s)
if m:
    print(f'after re.search(p, s), m.group() is "{m.group()}"')
```

```
s starts as "today 03/05/2011 is March 5th, 2011"
p starts as "\d{2}/\d{2}/\d{4}"
after re.search(p, s), m.group() is "03/05/2011"
```

By grouping parts of the pattern with `(` and `)`, the `group` function can give the part of the string matched by each group.

In [129…

```
import re
s = "on 03/05/2011 it rained"
print(f's starts as "{s}"')
p = "(\d{2})/(\d{2})/(\d{4})"
print(f'p starts as "{p}"')
m = re.search(p, s)
if m:
    print(f'after re.search(p, s), m.group() is "{m.group()}"')
    print(f'm.group(1) is the month "{m.group(1)}"')
    print(f'm.group(2) is the day "{m.group(2)}"')
    print(f'm.group(3) is the year "{m.group(3)}"')
```

```
s starts as "on 03/05/2011 it rained"
```

```
p starts as "(\d{2})/(\d{2})/(\d{4})"
after re.search(p, s), m.group() is "03/05/2011"
m.group(1) is the month "03"
m.group(2) is the day "05"
```

Files

Files hold data that programs use.

- Programs can read data from files.
- Programs can write data to files.
- Programs can add data to files that already are there.

`Open` and `Close` functions

The `open` function gets a program ready to read or write a file.

- `open(` *file-name* `)` gets the program ready to read a file.
- `open(` *file-name* `, 'w')` gets the program ready to write a file.
- `open(` *file-name* `, 'a')` gets the program ready to add to a file that already exists.

`open` returns a *file descriptor* used to read or write to the file. The `close` function is used complete reading or writing to a file descriptor.

`files.txt` contains these lines.

```
05/20/2021 $20.11 Office supplies
12/01/2021 $123.11 Desk chair
06/14/2021 $50.82 Telephone bill
```

The variable names `f`, `g`, and so on will be used for file descriptors.

In [130…

```
f = open("files.txt", "r")
for s in f:
  print(s.strip())
f.close()
```

```
05/20/2021 $20.11 Office supplies
12/01/2021 $123.11 Desk chair
06/14/2021 $50.82 Telephone bill
```

Examples

These are examples of programs that put together what we've learned.

File processing

One of the most common uses of computers is to process file data. Much of the data used in data processing starts as text that can be read in files. File processing programs read data in files, analyze it somehow, and save the data in some useful way.

Summarizing groups of records

The program below reads the office supply purchase data from the file `file.txt` used above and prints a summary of the purchases with the total cost.

This example uses regular expressions to separate the different items on each record. These are regular expressions to pull out the date, cost, and name of supplies.

- date `(\d{2})/(\d{2})/(\d{4})`
- cost `\$(\d+\.\d+)` where `\$` and `\.` are escaped characters because `$` and `.` are special characters
- name of supplies `(.*)` where `.*` matches all the text after the cost

`import re` is needed to start using regular expressions.

In [131…

```
import re
p = "(\d{2}/\d{2}/\d{4}) \$(\d+\.\d+) (.*)"
f = open("file.txt", "r")
total = 0.0
for line in f:
    m = re.search(p, line)
    print()
    print(line.strip())
    if m:
        date = m.group(1)
        cost = m.group(2)
        name = m.group(3)
        total += float(cost)
        print(f'date {date} cost ${cost} name "{name}"')
    else:
        print("    line is not the right format")
f.close()
print()
print(f'total cost: ${total}')
```

```
05/20/2021 $20.11 Office supplies
date 05/20/2021 cost $20.11 name "Office supplies"

12/01/2021 $123.11 Desk chair
date 12/01/2021 cost $123.11 name "Desk chair"

06/14/2021 $50.82 Telephone bill
date 06/14/2021 cost $50.82 name "Telephone bill"

total cost: $194.04
```

Summarizing groups of records by type

This example uses regular expressions to add the cost of items in the file and accumulating them by type. The example uses the file `orders.txt` , which is a data source for purchases of cell phone, cable TV, and internet services. This example Each line has the format:

- date
- customer
- cost
- service purchased

The program takes the date, name, cost, and service purchased and totals the purchases by name and service type. `orders.txt` contains these lines.

```
01/05/2022 Colin Heath $70.00 Phone-Cell
01/11/2022 Courtney Collins $14.99 Cable-HBO
01/12/2022 John Johnson $70.00 Phone-Cell
01/17/2022 Colin Heath $50.00 Internet-100MB
02/18/2022 John Johnson $80.00 Internet-1GB
02/21/2022 Courtney Collins $6.99 Cable-Hulu
```

```
02/23/2022 Colin Heath $6.99 Cable-Hulu
03/07/2022 Courtney Collins $50.00 Internet-100MB
```

These are regular expressions to pull out the date, cost, and name of supplies.

- date `(\d{2})/(\d{2})/(\d{4})`
- first name `(\w+)`
- last name `(\w+)`
- cost `\$(\d+\.\d+)` where `\$` and `\.` are escaped characters because `$` and `.` are special characters
- name of service `(.*)` where `.*` matches all the text after the cost

These are notes about the program.

- The dictionaries *customers* and *services* are used to hold the total cost for the purchases for each person and each service.
- The function `add_to` totals the cost of an item by customer and service.
- The first time the cost is assigned to the dictionary entry for the item, because the item is not yet in the dictionary. After that it adds the cost to the total for that dictionary item.
- The results are printed in a readable way using the string function `format`.
- `import re` is needed to start using regular expressions.

In [132…

```
import re

def print_totals(item, dict):
    keys = list(dict.keys())
    keys.sort()
    item_header = '{:17s}'.format(item)
    print(f'{item_header}  costs')
    print("-----------------  --------")
    for key in keys:
        name = key
        total = dict[key]
        print('{:17s}  ${:7.2f}'.format(name, total))

def add_to(customers, name, services, item, cost):
    if name in customers:
        customers[name] += cost
    else:
        customers[name] = cost
    if item in services:
        services[item] += cost
    else:
        services[item] = cost

p = "(\d{2}/\d{2}/\d{4}) (\w+) (\w+) \$(\d+\.\d+) (.*)"
f = open("orders.txt", "r")
customers = {}
services = {}
print('date         name                 service         cost')
print('------------ -------------------- --------------- --------')
for s in f:
    line = s.strip()
    m = re.search(p, line)
    if m:
        date = m.group(1)
        first = m.group(2)
        last = m.group(3)
        full_name = f'{last}, {first}'
        cost = m.group(4)
        cost_float = float(cost)
        service = m.group(5)
        print('{:12s} {:20s} {:15s} ${:7.2f}'.format(date, full_name, service
        add_to(customers, full_name, services, service, cost_float)
    else:
        print("    line is not the right format")
        print(line)
f.close()
print()
print(f'total cost for customers')
print_totals("customers", customers)
print()
print(f'total cost for services')
print_totals("services", services)
```

```
date         name                 service         cost
------------ -------------------- --------------- --------
01/05/2022   Heath, Colin         Phone-Cell      $  70.00
01/11/2022   Collins, Courtney    Cable-HBO       $  14.99
```

```
01/12/2022   Johnson, John        Phone-Cell      $  70.00
01/17/2022   Heath, Colin         Internet-100MB  $  50.00
02/18/2022   Johnson, John        Internet-1GB    $  80.00
02/21/2022   Collins, Courtney    Cable-Hulu      $   6.99
02/23/2022   Heath, Colin         Cable-Hulu      $   6.99
03/07/2022   Collins, Courtney    Internet-100MB  $  50.00

total cost for customers
customers          costs
-----------------  --------
Collins, Courtney  $  71.98
Heath, Colin       $ 126.99
Johnson, John      $ 150.00

total cost for services
services           costs
-----------------  --------
Cable-HBO          $  14.99
Cable-Hulu         $  13.98
Internet-100MB     $ 100.00
Internet-1GB       $  80.00
Phone-Cell         $ 140.00
```

Joining two data sources

Often program data comes in two or more data sources. These may be in files, or in *databases*, where the data are records containing different data items. Two data sources may have records that describe the same thing.

This example combines data describing movies in two files. The example uses the file `movies.txt` , which is a data source for movies and when they were released, and the file `actors.txt` , which is a data source for the actors in those movies.

Each line of `movies.txt` has the format:

- name of the movie
- date released

Each line of `actors.txt` has the format:

- name of an actor
- a movie they starred in

The program combines the data in both files by taking a movie name from `actors.txt` and finding the movie's release date from `movies.txt` , and printing one line with these data items.

- actor
- movie
- release date

The program pulls out the actor, movie, and release date from the records, but we want to allow spaces in the movie name so we separate the data items with `,` instead of a space.

`movies.txt` contains these lines.

```
Forrest Gump,1994
Kate & Leopold,1998
```

`actors.txt` contains these lines.

```
Meg Ryan,Kate & Leopold
Tom Hanks,Forrest Gump
```

The actors dictionary is this.

The movies dictionary is this.

The actors and movies dictionaries are joined by the movie.

In [133…

```
def print_actors(movies, actors):
    actor_names = list(actors.keys())
    actor_names.sort()
    print(f'actor         movie              release date')
    print("------------  -----------------  ------------")
    for actor in actor_names:
        movie = actors[actor]
        release_date = movies[movie]
        print('{:12s}  {:17s}  {:4s}'.format(actor, movie, release_date))

def add_actor(actors, line):
    items = line.split(",")
    actor = items[0]
    movie = items[1]
    actors[actor] = movie

def add_movie(movies, line):
    items = line.split(",")
    movie = items[0]
    release_date = items[1]
    movies[movie] = release_date

f = open("actors.txt", "r")
actors = {}
for s in f:
    line = s.strip()
    add_actor(actors, line)
f.close()

g = open("movies.txt", "r")
movies = {}
for s in g:
    line = s.strip()
    add_movie(movies, line)
g.close()

print_actors(movies, actors)
```

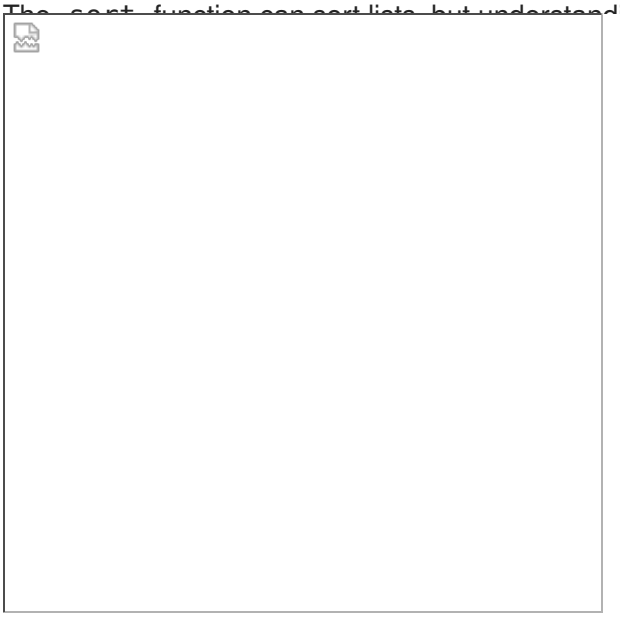
```
actor         movie              release date
------------  -----------------  ------------
Meg Ryan      Kate & Leopold     2001
Tom Hanks     Forrest Gump       1994
```

Data Structures

Composite data or *data structures* simplifies much of programming. Knowing how to program with algorithms using composite data is one of the basic skills of programming.

Sorting a list

The `sort` function can sort lists, but understanding how it works is a good example for



One way to sort the list is to start at the bottom and move the lowest number to the top. Here, 3 is swapped with 8, then 3 is swapped with 9.

The lowest number is now at the top. Next start at the bottom of the list and move the second highest number to be second in the list. Here, 8 is swapped with 9.

The second lowest number is in the second position and the list is now sorted.

The way we program this is to say what we want to do in English first.

- Start with a list *n* long
- *Bubble* the lowest item up to the top of the list
- Bubble the next lowest item up to the second item in the list
- Repeat this until we bubble the second highest number to the second-to-the-last item in the list

We compare the last item in the list to the one above it, and swap them if the last item is less. We do this until we compare the second item in the list to the first item in the list. We then compare the last item in the list to the one above it, and swap them if the last item is less. We compare and swap items until we compare to the second item in the list. This says that bubble up to the top of the list, to the second item of the list, until we compare to the second-to-the-last item in the list, or *n-1* times, because there are n-1 items between the top and the second-to-the-last item.

We want a program that looks like this:

```
sort(L):
  n = len(L)
  # bubble the lowest item to the top, then the second, down to
the second-to-last spot in the list
  for i in 1 to n-1:
    from j = n (the last item) to i-1 # the spot below the one
we're bubbling to
      compare the item at j to the item at j-1
```

```
swap them if the item at j is less than the item at j-1
```

That is about it, except shifting the spots in the list because lists positions start at 0, not 1. Here is the program as a function, with some prints to track what it's doing.

In [134…

```
def sort(L):
    n = len(L)
    print(f'at the start L is {L} and len(L) is {n}')
    for i in range(n-1):
        print(f'  bubbling the next lowest item to the position {i}')
        for j in range(n-1, i, -1):
            print(f'    comparing L[{j}] = {L[j]} to L[{j-1}] = {L[j-1]}')
            if (L[j] < L[j-1]):
                print(f'    swapping  L[{j}] = {L[j]} and L[{j-1}] = {L[j-1]}
                x = L[j]
                L[j] = L[j-1]
                L[j-1] = x
        print(f'    L at round {i} is {L}')

L = [9, 8, 3]
sort(L)
print(f'at the end L is {L}')
```

```
at the start L is [9, 8, 3] and len(L) is 3
  bubbling the next lowest item to the position 0
    comparing L[2] = 3 to L[1] = 8
    swapping  L[2] = 3 and L[1] = 8
    comparing L[1] = 3 to L[0] = 9
    swapping  L[1] = 3 and L[0] = 9
    L at round 0 is [3, 9, 8]
  bubbling the next lowest item to the position 1
    comparing L[2] = 8 to L[1] = 9
    swapping  L[2] = 8 and L[1] = 9
    L at round 1 is [3, 8, 9]
at the end L is [3, 8, 9]
```

The final list matches our diagram. The steps to write this program were:

- coming up with a small test case that shows how the program works
- thinking out an idea of how the program might work, here the key was *bubbling* the lowest element to it's right position each time
- making a diagram of steps as the program runs
- writing the idea in English in a way that the steps are understandable
- writing the Python instructions, printing out variables at various steps as the program runs

One way this program would be useful is that what we define as *lower* items might be more complicated than *the number value of the item is less*. The items might be strings, or *less* might be that a "mouse" might be less than a "rabbit", a "rabbit" might be less than a "dog", and so forth. We would change the comparison `L[j] < L[j-1]` to our own comparison to test if an item is *less* than another. We saw how `for` loops can control the bubbling, and that a small program can sort a list of any size.

Data Cleaning

A common task in Data Science is cleaning raw data to make it usable for further analysis. Following are types of data that cleaning will correct or remove.

- incorrectly formatted
- duplicate fields or lines
- inconsistent, non-standard, or mislabeled categories or classes
- corrupted, invalid, inaccuate, irrelevant, outliers, or incomplete
- has typos, misspellings, or syntax errors
- missing codes, data, or fields

Removing duplicate lines in a file

A common task in data cleaning is removing duplicates in files. In this example we have a file with duplicate lines, and we want to remove the duplicates so we just have unique lines in the file. In this case we have a sample data file with some duplicate lines. Before removing duplicate lines, the file may need to be *sorted* first. The program idea is that we look at the *previous line* to decide if a line is a duplicate:

- case 1: the previous line is the same, then the current line is a duplicate
- case 2: the previous line is not the same, then the current line is new and should be kept (printed back out)

We then see there are two more cases:

- case 3: the first line in the file. There is no previous line, so the current line is new and should be kept.
- case 4: we run out of lines in the file. There is no current line, so there is nothing we have to do.

We can see in this case that:

- the first line is A, line 0 (we always count from 0, this is case 3 so we print it
- the second line, line 1, is B, this is case 2 so we print it
- the third line, line 2, is B, this is case 1 and we ignore it
- the fourth line, line 3, is C, this is case 2 so we print it
- the fifth line, line 4, is D, this is case 2 so we print it
- the sixth line, line 4, is D, this is case 1 so we ignore it

We can now write the program. `last` is the last line we saw. `first` tells us whether we are at the first line of the file, which is case 3. We read all the lines of the file, and don't need to do

anything at the end, just close the file.

Let's say the file `duplicates.txt` is this:

```
A
B
B
C
D
D
```

This is the program. `strip` is used because the line actually has a `newline` , "\n", at the end which we can ignore. The line beginning with `#` is a *comment*. It is for notes we make to about what the program is doing, it is ignored when the program runs. The first time we read a line, `first` is `True` , we have case 3 and we print the line. We use `elif` to test the line after the first is read and `first` is set to `False` . If the test is true, the line is different from the last and we have case 2, and we print the line. If the test is false, the line is the same as the last and we have case 1, and we ignore the line.

In [135…

```
first = True
f = open("duplicates.txt", "r")
for line in f:
    current = line.strip()
    if first:
        first = False
        last = current
        print(current)
    elif current != last:
        print(current)
    # otherwise, if current == last it is a duplicate and we ignore it
    last = current
f.close()
```

```
A
B
C
D
```

Another way to remove duplicate lines

If the purpose is to remove duplicates and the order of the lines do not matter, this is another way to remove duplicate lines. A dictionary has only unique keys. When a dictionary is first assigned with a key, a new entry is created. The second time the dictionary is assigned with the key, no new entry is created but the previous value is replaced. By using the line that is read as the key, they keys will only keep one copy of it. We can read the file, assign a dictionary entry with the line as the key with any value, and print the keys at the end.

In this Let's say the file `duplicates.txt` is this:

```
D
D
```

```
A
C
B
B
```

In this case the lines with the duplication removed come out in the same order, but that is not guaranteed. This is show as this.

In [136…

```
f = open("unordered-duplicates.txt", "r")
unique_lines = {}
for line in f:
    current = line.strip()
    unique_lines[current] = 1
f.close()
print('unique lines:')
for line in unique_lines.keys():
    print(line)
```

```
unique lines:
D
A
C
B
```

Fixing inconsistent codes

Another data cleaning task is replacing inconsistent spelling with one recognizable value. "Not applicable" may be used as "NA" or "N/A". "Drive" may be "Dr." or "Dr". A dictionary can have the incorrect spelling as the key and the correct spelling as the value.

Let's say the file `misspellings.txt` is this:

```
Smith,John,N/A,230 Overland Dr.
Jones,Michael,NA,34 Blue Ridge Drive
Lund,Mary,Not applicable,Main St
```

In [137…

```
misspellings = {"N/A": "NA", "Not applicable": "NA", "Drive": "Dr", "Dr.": "Dr
f = open("misspellings.txt", "r")
for line in f:
    current = line.strip()
    for term in misspellings.keys():
        current = current.replace(term, misspellings[term])
    print(current)
f.close()
```

```
Smith,John,NA,230 Overland Dr
Jones,Michael,NA,34 Blue Ridge Dr
Lund,Mary,NA,Main Dr
```

Numbers

One of the most useful functions of computers is working with numbers. Computers are designed to handle calculations that are difficult for people.

Roman Numerals

The problem is converting Roman numerals like `III` to 3. The Roman up to v look like this.

```
4    3    2    1
IV   III  II    I
```

We can use regular expressions to recognize these. We can show one to three, which are `I`, `II`, or `III`, as 1, 2, or 3 `I`'s. This can match the regular expression `I{1,3}`. We can test this using `re.search`.

In [7]:
```
import re
p = "I{1,3}"
m = re.search(p, "I")
print(f're.search(p, "I") is "{m.group()}"')
m = re.search(p, "II")
print(f're.search(p, "II") is "{m.group()}"')
m = re.search(p, "III")
print(f're.search(p, "III") is "{m.group()}"')
```

```
re.search(p, "III") is "III"
re.search(p, "II") is "II"
re.search(p, "I") is "I"
```

Then we also see that we have either 4, which is `IV`, *or* 1-3, which are `I`, `II`, or `III`. We show this with `|`, or *alternation*.

```
  4 or 3-1
IV|I{1,3}
```

We can test this using `re.search`.

In [10]:
```
import re
p = "IV|I{1,3}"
m = re.search(p, "I")
print(f're.search(p, "I") is "{m.group()}"')
m = re.search(p, "II")
print(f're.search(p, "II") is "{m.group()}"')
m = re.search(p, "III")
print(f're.search(p, "III") is "{m.group()}"')
m = re.search(p, "IV")
print(f're.search(p, "IV") is "{m.group()}"')
```

```
re.search(p, "I") is "I"
re.search(p, "II") is "II"
re.search(p, "III") is "III"
re.search(p, "IV") is "IV"
```

We can use a dictionary to get the values for each Roman numeral group.

```
D = {"IV": 4, "III": 3, "II": 2, "I": 1}
```

We can now put this together and test with `re.search` to get the value of all Roman numbers between one and four.

In [19]:

```
import re
D = {"IV": 4, "III": 3, "II": 2, "I": 1}
p = "IV|I{1,3}"
m = re.search(p, "I")
print(f're.search(p, "I") is "{m.group()}" and the value is {D[m.group()]}')
m = re.search(p, "II")
print(f're.search(p, "II") is "{m.group()}" and the value is {D[m.group()]}')
m = re.search(p, "III")
print(f're.search(p, "III") is "{m.group()}" and the value is {D[m.group()]}'
m = re.search(p, "IV")
print(f're.search(p, "IV") is "{m.group()}" and the value is {D[m.group()]}')
```

```
re.search(p, "I") is "I" and the value is 1
re.search(p, "II") is "II" and the value is 2
re.search(p, "III") is "III" and the value is 3
re.search(p, "IV") is "IV" and the value is 4
```

Computing a square root

Python has a `sqrt` function to compute a square root of a number.

In [138…

```
import math
x = int(input('Give the number you want the square root of:'))
print(f'the square root of {x} is {math.sqrt(x)}')
```

```
the square root of 7 is 2.6457513110645907
```

`sqrt` is one of the functions Python supplies for common tasks to save us time programming. `import math` says we will use the `math` *module* where a module is a group of functions. We call `math.sqrt` to tell Python that `sqrt` is part of the `math` module.

We can compute the square root ourselves by using a formula. If we make a *guess* of the square root and divide the number by it, if the guess is right we will get the same number back.

In [139…

```
x = 9
y = 9/3

print(f'making a guess that 3 is the square root of {x}, the result of dividi
```

```
making a guess that 3 is the square root of 9, the result of dividing 9 by 3 i
s 3.0
```

If the guess is wrong, the result of dividing will be different, but the right guess will be somewhere between the two.

We'll try an idea for the program.

- If we have a number, we make a guess and divide it into the number.
- If the guess and the result of dividing it into the number are the same, we've found the square root.
- If they are not the same, we can try a new guess that is the number halfway between the

guess and the result.

The number halfway between two numbers is the average, or $\frac{guess + \frac{guess}{number}}{2}$.

In [140…

```
x = 9
y = 2
z = x/y
print(f'dividing {x} by 2 gives {y}, and the average of the two is {(y + z) /
```

```
dividing 9 by 2 gives 2, and the average of the two is 3.25
```

It turns out the second step is not as easy as we think. The guess will almost always be a real number, and those two numbers might be very close but still not the same. Instead of the being the *same*, we will test whether their difference is less than some small number, say `0.001`. We'll also use some different variable names to try to keep track of things better. We'll try half of the number as the first guess.

In [141…

```
import math
x = int(input('Give the number you want the square root of:'))
guess = x / 2
next_guess = (x / guess) / 2
tries = 0
while abs(guess - next_guess) > 0.001:
    tries += 1
    guess = next_guess
    next_guess = (guess + x / guess) / 2
    print(f'try {tries}, the guess is {round(guess, 4)}, the next guess is {r
print(f'the final guess is {round(guess, 4)} and the square root from math.sq
```

```
try 1, the guess is 1.0, the next guess is 4.0, and the difference is 3.0
try 2, the guess is 4.0, the next guess is 2.875, and the difference is 1.125
try 3, the guess is 2.875, the next guess is 2.6549, and the difference is 0.2
201
try 4, the guess is 2.6549, the next guess is 2.6458, and the difference is 0.
0091
try 5, the guess is 2.6458, the next guess is 2.6458, and the difference is 0.
0
the final guess is 2.6458 and the square root from math.sqrt(7) is 2.6458
```

Try a large number and see how many tries it takes. It usually won't take that many. The program did not take many instructions.

Finding the factors of a number

The factors of a number are those that divide evenly into the number. If we have a factor, then the number divided by the factor has a remainder of `0`. We can find the remainder with the *modulo* operator `%`.

We can test that we have all the factors in a `while` loop. We know we have all the factors when dividing all of them into the number gives `1`. A factor may occur more than once, so we

need a `while` loop inside the `while` loop to test each factor more than once. '1' is always a factor of the number, so we'll start testing with `2`.

In [142…

```
number = int(input('Give the number you want the factors of:'))
factor = 2
factors = []
x = number
while (x != 1):
    while x % factor == 0:
        factors.append(factor)
        x = x / factor
    factor += 1
print(f'the factors of {number} are {factors}')
```

```
the factors of 36 are [2, 2, 3, 3]
```

Finding prime numbers

The only factors a prime numbers have are 1 and the number. Prime numbers are very useful for cryptography and are good to know. We will use the factor example above as a *function* to find this. A number is prime when there are only two factors, 1 and the number itself. You will give us the largest number to test as prime

In [143…

```
def factors(number):
    factors = []
    factor = 2
    x = number
    while (x != 1):
        while x % factor == 0:
            factors.append(factor)
            x = x / factor
        factor += 1
    return factors

largest_prime = int(input('Give the largest prime number you want the search 
primes = []
for i in range(2, largest_prime + 1):
    factors_of_i = factors(i)
    if len(factors_of_i) == 1:
        primes.append(i)
print(f'the prime numbers up to of {largest_prime} are {primes}')
```

```
the prime numbers up to of 40 are [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]
```

Another way to test for prime numbers is the *sieve of Erathsenes*, named after the person who invented the method. You start by assuming all numbers up to the largest number are prime. For numbers less than the prime, you know that two times the number, three times the number, and so on up to the largest prime are not prime because they have at least two factors. For each number, you then go through the list and mark the number as not prime. You do not need to test even numbers after 2, because 2 will mark all even numbers after it. We use a list comprehension to create a list of True's las long as the largest prime number. We will set each

element to False that is a multiple of the number being tested

In [144…

```
largest_prime = int(input('Give the largest prime number you want the search 
numbers = [True for i in range(largest_prime)]
for i in range(2, largest_prime):
    next_nonprime = i * 2
    while next_nonprime <= largest_prime:
        numbers[next_nonprime - 1] = False
        next_nonprime += i
primes = []
for i in range(1, largest_prime):
    if numbers[i]:
        primes.append(i + 1)
print(f'the prime numbers up to {largest_prime} are {primes}')
```

```
the prime numbers up to 40 are [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]
```

Try finding the primes up to 30000 with both programs. The Sieve of Erasthenes should be a lot faster

Glossary

Algorithms are general designs for common program tasks, such as organizing, searching for, or transforming data.

Data are terms, facts, measurements, or statistics that when processed become *information*.

CPU is the computer *central processing unit* that runs low-level machine instructions.

Databases hold data much like text files, except that databases can hold types of data other than text, such as images. Databases also usually can read records based on the data in them, such as names, dates, or any other part of the record. These are called *keys*.

Information is data that is preprocessed, organized, analyzed, and interpreted.

Lowercase letters are characters from `a` to `z`.

Newline is a character that is not visible, but causes the printing to start on a new line.

Text is letters, numbers, and punctuation much as you would type on a keyboard. Text may include European letters like ö, é, î and ñ.

Uppercase letters are characters from `A` to `Z`.

A *web browser* is a program to navigate the Internet, such as Chrome, Edge, Firefox, or Safari.